

위키

≡ 태그

B+ Tree project

과제 명세

`program -c index_file b` : 빈 인덱스 파일 생성

`program -i index_file data_file` : data file 에 있는 key-value 을 하나씩 읽어서 B+Tree 에 삽입하는것 (하나씩 삽입 연산으로 점차적 구성, 노드들이 생성되어서 트리구조 형성) 이 때 노드 정보들이 정해지고 이를 index file 에 저장

`program -d index_file data_file` : B+Tree 에 이미 데이터가 삽입되어져 있는 상태에서 해당 파일에 있는 키들을 삭제하는 것 (이때 트리 재조정되어서 노드 정보가 바뀌면 인덱스 파일에 수정사항 저장)

프로그램을 실행하면 인덱스 파일에 B+트리 기반으로 노드 정보를 파일에 저장

인덱스 파일 안에는 비벨 트리의 각 노드가 저장되어있고, 인덱스 파일에 저장된 노드를 메모리에 불러오고 사용하는 것. (노드 안에 각 정보가 들어있음)

Index file

1. 인덱스 파일 개념

- 인덱스 파일 : 디스크에 저장된 파일로 B+ 트리의 모든 노드를 저장
- 파일 내 위치 (오프셋) : 노드 간 연결은 파일 내의 위치를 이용해 관리함
- 인덱스 파일은 바이너리 파일 구조

2. 인덱스 파일의 구조

- 파일 헤더

- tree 의 메타 데이터 (b, 루트 노드 위치)
- 루트 노드 오프셋: 트리의 루트 노드가 파일 내에서 어디에 저장되어 있는지 가리키는 포인터 (파일 내 위치)
- 노드 데이터
 - Non-leaf node
 - ✓ *m*: # of keys
 - ✓ *p*: an array of<key, left_child_node> pairs
 - ✓ *r*: a pointer to the rightmost child node
 - Leaf node
 - ✓ *m*: # of keys
 - ✓ *p_*: an array of<key, value(or pointer to the value)> pairs
 - ✓ *r*: a pointer to the right sibling node
- 노드 타입 : 노드가 리프 노드인지 논리프 노드인지 나타내는 플래그 저장
- 노드 크기 : b , 각 노드는 최대 b-1 개의 키와 최대 b 개의 자식을 가질 수 있음
- 키 리스트 : 노드 내에 저장된 키 값들의 리스트
- non-leaf : 포인터 리스트 자식 노드에 대한 포인터 (파일 내 위치)
- leaf : 데이터에 대한 포인터 또는 실제 데이터를 저장하고 다음 리프 노드에 대한 포인터 (오른쪽)

파일 위치 (오프셋)

- **파일 포인터** : 파일을 읽거나 쓸 때, 파일 내의 특정 위치 (오프셋) 를 가리키는 파일 포인터가 있음
 - 파일 내에서 현재 읽거나 쓸 위치를 나타냄
 - 위치 계산 : 노드를 파일에 쓸 때마다, 그 노드가 파일의 어느 위치에 기록되었는지 파일 포인터를 통해 알 수 있음
 - 이 위치 (오프셋)을 저장해두고 나중에 이 위치로 돌아가서 그 노드를 읽을 수 있음
- 직접 파일 위치를 관리하지 않고, 프로그램 내에서 자동으로 계산된 파일 포인터를 이용해 관리하는 방식

1. 루트 노드를 파일에 저장

루트 노드를 저장하기 전, 파일의 현재 위치를 확인 (`file.tell()`)

그 위치가 루트 노드의 위치가 됨

해당 위치를 파일의 헤더 (0) 에 저장함

2. 루트 노드 읽어오기

파일을 다시 열고 파일의 헤더에서 루트 노드 포인터를 읽어옴

이 포인터로 파일의 해당 위치로 이동해서 루트 노드를 메모리로 불러옴 (`pickle.load`)

Implementation

node

→ 고정된 크기의 노드 구조

오프셋이 꼬이지 않게 하기 위해서 (예를 들어, 키가 추가되는 경우 아래에 있던 노드를 덮어씌울 위험이 존재함) 각 노드가 최대 b 개의 자식을 가질 수 있는 것으로 미리 노드의 크기를 지정하여 사용했다.

```
class Node:
    def __init__(self, is_leaf, b, position=None):
        self.is_leaf = is_leaf # leaf node flag
        self.b = b # 최대 자식 개수 (최대 키 개수 : b-1)
        self.m = 0 # 현재 키 개수
        self.key = [-1] * (b-1)
        self.leftchild = [-1] * (b-1) # -1: none
        self.rightchild = -1 # 노드의 가장 오른쪽 자식 포인터 (leaf : sibling )
        self.position = position # 노드의 바이너리 파일 내 위치 (오프셋으로 사용)
```

→ 리프/ 논리프 노드 모두 같은 구조를 갖는데, 안에 들어있는 데이터의 차이점은 아래와 같다.

- 리프: leftchild: 왼쪽 자식 포인터, rightchild: 오른쪽 자식 포인터
- 논리프 : leftchild : value, rightchild: 오른쪽 형제 리프 노드

- 입력받은 b 에 따라 고정 크기를 가지는 노드를 index file 에 쓰고 load 하는 형식
- 또한 노드에 변경사항이 생기면 update 를 해줘서 index 파일에 다시 써줬음

Create

`program -c index_file b` : 빈 인덱스 파일 생성

```
if cmd == '-c':  
    b = int(sys.argv[3])  
  
    tree = BPTree(index_file, b)  
    tree.create_index_file()
```

빈 트리를 생성한다.

```
def __init__(self, is_leaf, b, position=None):  
    self.is_leaf = is_leaf # leaf node flag  
    self.b = b # 최대 자식 개수 (최대 키 개수 : b-1)  
    self.m = 0 # 현재 키 개수  
    self.key = [-1] * (b-1)  
    self.leftchild = [-1] * (b-1) # -1: none  
    self.rightchild = -1 # 노드의 가장 오른쪽 자식 포인터 (leaf  
: sibling )  
    self.position = position # 노드의 바이너리 파일 내 위치 (오프셋으로 사용)  
  
def get_node_size(b):  
    # 고정된 노드의 크기 계산 -> 매번 이 크기로 할당해줘야함  
    return struct.calcsize(f'<?ii{b - 1}i{b - 1}iii')  
  
def __init__(self, index_file, b=None):  
    self.index_file = index_file  
    self.b = b  
    self.node_cache = {}  
  
    if b is not None: # create  
        self.node_size = Node.get_node_size(b) # 고정된 노드  
크기
```

```

        else: # create 가 아니면, 기존 인덱스 파일을 사용해야함
            self.load_index_file()

    def create_index_file(self):
        with open(self.index_file, 'wb') as file:
            # header 작성 , header = b, root node position
            header = struct.pack('ii', self.b, -1) # 최초 루트 노드
            위치는 -1 로 임의 설정
            file.write(header)

```

이때 입력받은 b 를 이용해서 앞으로 생성될 노드의 고정 사이즈를 지정해두고,
create 를 하게되면 이후에 index file 참조를 위한 메타 데이터를 포함한다.

메타 데이터에는 다음과 같은 정보를 포함한다.

- b : size of each node (max. # of child nodes) [int]
- root node position : 이후 트리 참조를 위해 루트 노드 위치 (오프셋) 을 저장해 두기 위해 자리를 만들어 둔다. [int]

Insert

Case 1. 분할이 일어나지 않는 경우

→ 첫번째 리프 노드를 제외하고는 항상 리프 노드의 두번째 키 이후부터 삽입되기 때문에
인터널 노드의 인덱스를 신경쓰지 않아도 됨

Case 2. 분할- 리프 노드 (루트노드가 아닌 경우)

1. mid key ($b//2$)
2. left child node 새로 만들기 (original node 에서 mid key 전까지 밀어넣기)
left child node 의 right ptr은 original node 로 연결
(optional) right ptr 로 original node 를 가리키고 있던 리프 노드가 있다면 해당 노드의 right ptr 을 left child node 로 연결
3. original node 에 mid key 를 포함한 오른쪽 키들만 남기기 (부모노드의 right ptr 혹은 해당 노드를 가리켜야 하는 키의 left ptr 은 어차피 original node 를 가리킴 → 신경 x)
4. mid key 의 leftptr 을 left child node 로 연결
5. mid key & left ptr 을 부모 노드에 삽입

Case 3. 분할 - 리프 노드 (루트 노드 인 경우)

- case 2 에 +
1. 루트 노드 만들기
 2. mid key & left ptr 을 루트 노드에 추가
 3. 루트 노드의 right ptr 을 original node 로 연결
 4. 헤더 갱신

Case 4. 인터널 노드

1. mid key 뽑기 $b//2$
2. left child node 새로 만들기 (original node 에서 mid key 전까지)
right ptr 을 mid key 의 left ptr 로 연결해줌
3. original node 를 mid key 를 제외한 오른쪽만 남기기
4. mid key 의 left child 를 left child node 로 연결하고 부모 노드에 추가

if 부모 노드가 없다면 ? 루트 노드 였던 것

- root 노드 만들고 right ptr 을 original node 로 연결하는 것 추가
- 헤더 갱신

```
if cmd == '-i':
    input_file = sys.argv[3]
    tree = BPTree(index_file)
    with open(input_file, 'r') as file:
        input_data = csv.reader(file)
        for input in input_data:
            root_node = tree.load_root_node()
            tree.insert(input[0], input[1])
```

1. input 파일을 한줄씩 insert 에 넘겨준다. input[0] : key , input[1] : value

```
def insert(self, key, value):
    key = int(key)
    value = int(value)
```

```

root_node = tree.load_root_node()
if root_node is None:
    root_node = tree.create_node(is_leaf=True)
    tree.update_root_position(root_node.position)

insert_node = self.search(key, root_node)
insert_node.add_key(self, key, value)

```

2. input 을 위해 root node 를 로드해오고, (루트가 없다면 최초 생성한다.)

루트 노드에서부터 search 를 통해 insert 할 리프 노드를 찾는다. 이후 해당 노드에 키 (key, value) 를 추가한다.

```

def add_key(self, tree, key, value, is_split=False):
    # 노드가 가득 찼고 분할을 아직 하지 않은 경우
    if not is_split and (self.m >= self.b - 1):
        return self.split(tree, key, value)

    # 키를 삽입할 위치 찾기
    insert_idx = 0
    while insert_idx < self.m and self.key[insert_idx] < key:
        insert_idx += 1
    # 중복된 키가 있는지 확인
    if insert_idx < self.m and self.key[insert_idx] == key:
        print("중복된 키 삽입", key)
        return
    # 오른쪽으로 키를 이동하여 공간을 확보
    for i in range(self.m, insert_idx, -1):
        self.key[i] = self.key[i - 1]
        self.leftchild[i] = self.leftchild[i - 1]
    # 새로운 키와 값을 삽입
    self.key[insert_idx] = key
    self.leftchild[insert_idx] = value
    self.m += 1 # 현재 키의 개수를 업데이트
    # 분할이 아닌 경우에만 업데이트
    if not is_split:
        tree.update_node(self) # 현재 노드가 변경됨

```

```

def split(self, tree, key, value):

    # split 하기 전에 임의로 배열 크기 증가시켜놓음 (고정 크기 +1) -
    > 노드 업데이트 할 때는 줄여져야함 (확인)
    self.key.append(-1)
    self.leftchild.append(-1)

    self.add_key(tree, key, value, is_split=True)

    mid_idx = (self.b)//2
    mid_key = self.key[mid_idx]

    # 추가
    mid_key_leftchild = self.leftchild[mid_idx]

    # 리프 노드인 경우
    if self.is_leaf:
        # left child node 만들기
        # original node 의 처음 ~ mid 전까지 삽입
        new_node = tree.create_node(is_leaf=True)

    # 왼쪽 자식 노드의 키
    left_key = [-1] * (self.b-1)
    left_leftchild = [-1] * (self.b-1)
    for i in range(mid_idx):
        left_key[i] = self.key[i]
        left_leftchild[i] = self.leftchild[i]

    right_key = [-1] * (self.b-1)
    right_leftchild = [-1] * (self.b-1)
    for i in range(mid_idx, len(self.key)):
        right_key[i-mid_idx] = self.key[i]
        right_leftchild[i-mid_idx] = self.leftchild[i]

    # original node 가 오른쪽 자식 노드가 됨
    self.key = right_key
    self.leftchild = right_leftchild
    self.m = len([key for key in right_key if key != -1])

```

```

# 새로운 노드 == 왼쪽 자식 노드
new_node.key = left_key
new_node.leftchild = left_leftchild
new_node.m = len([key for key in left_key if key != -
1])
# 왼쪽 자식 노드(leaf)의 rightchild는 오른쪽 자식 노드(leaf)(s
elf) 를 가리킴
new_node.rightchild = self.position

# 왼쪽 자식 노드와 오른쪽 자식 노드 (지금은 현재 노드) 가 변경됨 -
> 업데이트
tree.update_node(new_node)
tree.update_node(self)

# range search
# 분할 후 midkey 값과 midleftchild(새로만든 왼쪽 자식 노드) 값
을 부모 노드로 올려줘야함
parent_node = self.load_parent_node(tree, self)
# print("\nparent node is\n", parent_node)
# 부모노드가 있다면 (루트 노드가 아닌 경우)
if parent_node:
    # # original 노드를 right child 로 가리키던 leaf 가 있다
    면, new node 로 연결해줘야함
    left_sibling = self.find_leftsibling(tree, self)
    if left_sibling:
        #("left sibling\n", left_sibling)
        left_sibling.rightchild = new_node.position
        tree.update_node(left_sibling)

    parent_node.add_key(tree, mid_key, new_node.position)
    tree.update_node(parent_node) # parent 노드 변경됨 ->
업데이트

    return # return 값 필요?

else: # 현재가 루트 노드인 경우
    new_root = tree.create_node(is_leaf=False) # 새로운 루

```

트 노드 만들고

```
tree.update_root_position(new_root.position)
```

키 값 추가후

```
new_root.add_key(tree, mid_key, new_node.position) #
```

를 현재 노드로 설정

```
tree.update_node(new_root)
```

인터널 노드인 경우

```
else:
```

left child node 만들기

original node 의 처음 ~ mid 전까지 삽입

```
new_node = tree.create_node(is_leaf=False)
```

왼쪽 자식 노드의 키

```
left_key = [-1] * (self.b-1)
```

```
left_leftchild = [-1] * (self.b-1)
```

```
for i in range(mid_idx):
```

```
    left_key[i] = self.key[i]
```

```
    left_leftchild[i] = self.leftchild[i]
```

```
right_key = [-1] * (self.b-1)
```

```
right_leftchild = [-1] * (self.b-1)
```

```
for i in range(mid_idx+1, len(self.key)):
```

```
    right_key[i-mid_idx-1] = self.key[i]
```

```
    right_leftchild[i-mid_idx-1] = self.leftchild[i]
```

original node 가 오른쪽 자식 노드가 됨

```
self.key = right_key
```

```
self.leftchild = right_leftchild
```

```
self.m = len([key for key in right_key if key != -1])
```

새로운 노드 == 왼쪽 자식 노드

```
new_node.key = left_key
```

```
new_node.leftchild = left_leftchild
```

```
new_node.m = len([key for key in left_key if key != -
```

```

1])
    # 왼쪽 자식 노드(leaf)의 rightchild는 mid_key 의 왼쪽 자식 노
    드를 가리킴
    new_node.rightchild = mid_key_leftchild

    tree.update_node(new_node)
    tree.update_node(self)

    mid_key_leftchild = new_node.position
    # 분할 후 midkey 값과 midleftchild(새로만든 왼쪽 자식 노드) 값
    을 부모 노드로 올려줘야함
    parent_node = self.load_parent_node(tree, self)

    # 부모노드가 있다면 (루트 노드가 아닌 경우)
    if parent_node:
        parent_node.add_key(tree, mid_key, mid_key_leftchild)
        tree.update_node(parent_node) # parent 노드 변경됨 ->
        업데이트

    return # return 값 필요?

    else: # 현재가 루트 노드인 경우
        new_root = tree.create_node(is_leaf=False) # 새로운 루
        트 노드 만들고
        tree.update_root_position(new_root.position)

        new_root.add_key(tree, mid_key, mid_key_leftchild) #
        키 값 추가후

        new_root.rightchild = self.position # 오른쪽 자식 노드
        를 현재 노드로 설정
        tree.update_node(new_root)

```

→ 위에서 작성한 알고리즘에 해당하는 add_key, split 함수

delete

진행 상황:

1. 최소 키 만족할 경우 (index 있을때 없을 때)-> 인덱스 삭제해
줌

2. 최소키 만족하지 않는 경우

2-1. left sibling 여유 키 있을 때 빌려오기

2-2. right sibling 여유 키 있을 때 빌려오기

case 1. 삭제 후 최소 키 조건을 만족할 경우 (인터널 노드에 지울 키 값 (인덱스) 가 존재하는 경우와 없는 경우)

- 리프 노드에서 키 값을 삭제
- 인터널 노드에 지울 키 값이 존재한다면 해당 키 값을 삭제해주고 오른쪽자식에서 가장 작은 키 값을 대신 채워줌

case 2. 삭제 후 최소 키 조건을 만족하지 않는 경우

case2-1. left sibling node에 여유 키가 있는 경우

- left sibling node 에서 가장 큰 키 (가장 오른쪽 키) 를 빌려옴 (left sibling node 에서 는 삭제해주기)
- 해당 키를 현재 삭제할 키의 부모 노드에 삽입
- 빌려온 키를 현재 노드에 추가해줌

case 2-2. right sibling node 에 여유 키가 있는 경우

- right sibling node 에서 가장 작은 키 (가장 왼쪽 키) 를 빌려옴 (right sibling node 에서는 삭제)
- 해당 키를 현재 삭제할 키의 부모 노드에 넣어줌
- 빌려온 키를 현재 노드에 추가해줌

case 3. 자식 노드 둘 다에게 여유키가 없는 경우 → merge

case 3-1. 만약 왼쪽 형제 노드가 있으면 왼쪽 형제 노드와 병합

1. 원래 노드 (child node) 에 병합 (right child 조정 필요 없음)
→ child-node 에 왼쪽 형제 노드 (left sibling) 키를 add 하면 됨 (key, value)
2. 부모에서 키 삭제
→ parent_node[idx-1] key, left child 삭제
→ parent node 를 idx-1 부터 한칸씩 당겨야함 (left child, key)
→ 원래 left child 를 가리키던 right pointer 조정 (child node 로)

case 3-2. 왼쪽 형제 노드가 없는 경우 오른쪽 형제 노드와 병합

1. 오른쪽 형제 노드 쪽에 병합
2. 부모 노드의 idx key, left child 삭제
3. 원래 노드 child node 를 가리키던 right pointer 를 오른쪽 형제 노드를 가리키게 함

→ 현재 딜리트는 재구성 및 부모 노드의 키 부족은 완성하지 못함

Instructions for Compile

Create

```
base ~/desktop/univ./4-2/database/btree  
python bptree.py -c index.dat 3
```

`python bptree.py -c index_file d` : 빈 인덱스 파일 생성

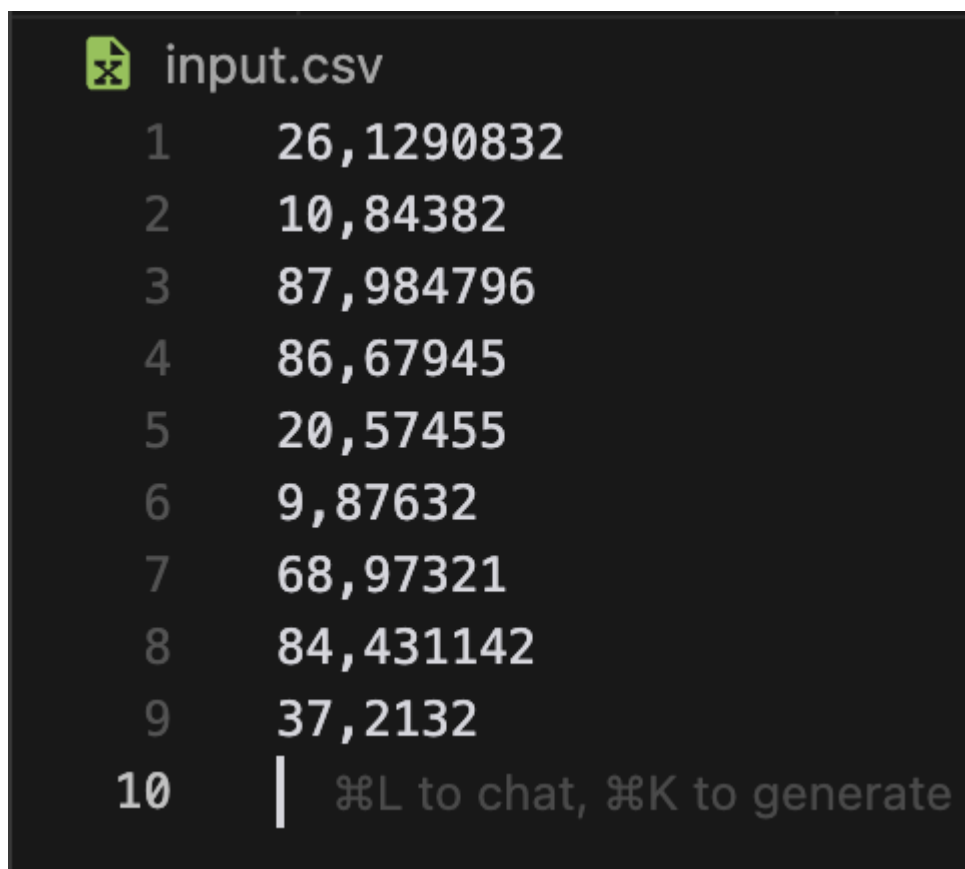
Insert

```
base ~/desktop/univ./4-2/database/btree git:(ma
python bptree.py -i index.dat input.csv
```

`python bptree.py -i index_file input_file` : insert 수행

** 오류 방지를 위해 , 비어있는 줄이 두 줄이 되지 않게 해주셔야 합니다.

** 마지막줄에서 한번 탭 해주세요. (아래 사진처럼 되게 해주시면 됩니다.)



```
input.csv
1      26,1290832
2      10,84382
3      87,984796
4      86,67945
5      20,57455
6      9,87632
7      68,97321
8      84,431142
9      37,2132
10     |  ⌘L to chat, ⌘K to generate
```

Single key search

```
base ~/desktop/univ./4-2/database/btree git:(
```

```
python bptree.py -s index.dat 38
```

```
26
```

```
68,86
```

```
NOT FOUND
```

```
base ~/desktop/univ./4-2/database/btree git:(
```

```
python bptree.py -s index.dat 37
```

```
26
```

```
68,86
```

```
2132
```

```
python bptree.py -s index_file key
```

→ 트리에 존재하지 않는 경우에는 "NOT FOUND" 출력

→ 존재하는 경우 타고 온 노드와 각 노드의 모든 키와, (노드가 바뀌는 경우 줄바꿈)

→ 마지막엔 value 값 프린트

Range Search

```
base ~/desktop/univ./4-2/database/btree gi
```

```
python bptree.py -r index.dat 20 68
```

```
20,57455
```

```
26,1290832
```

```
37,2132
```

```
68,97321
```

```
python bptree.py -r index.dat start_key end_key
```

range 에 해당하는 모든 key, value 프린트

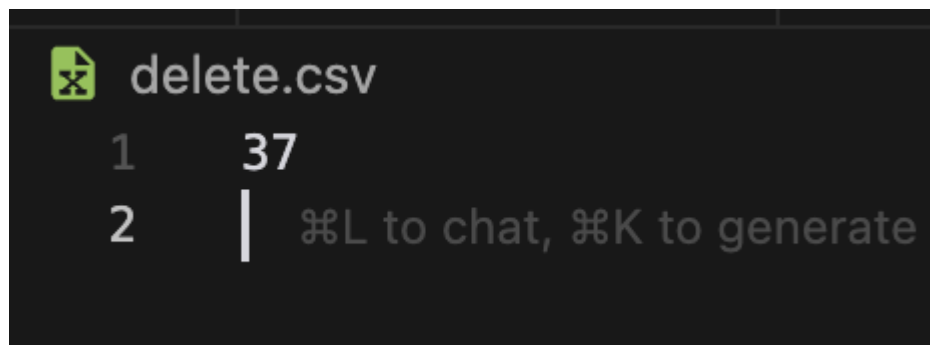
delete

```
base ~/desktop/univ./4-2/database/btree git:(main)  
python bptree.py -d index.dat delete.csv
```

```
python bptree.py -d index_file delete_file
```

** input.csv 와 마찬가지로 공백 줄이 두 줄이면 오류가 납니다.

** 혹시 모르니 마지막키에서는 탭 해주시면 감사드리겠습니다. 아래 사진과 같으면 됩니다.



```
delete.csv  
1 37  
2 | L to chat, K to generate
```

TestCase01



input.csv

```
1 26,1290832
2 10,84382
3 87,984796
4 86,67945
5 20,57455
6 9,87632
7 68,97321
8 84,431142
9 37,2132
10 | ⌘L to chat, ⌘K to generate
```



→ 정답 트리 구조는 다음과 같습니다. 아래에서 비교하겠습니다.

```

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.118s)
python bptree.py -c index.dat 3

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.082s)
python bptree.py -i index.dat input.csv

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.076s)
python bptree.py -ph index.dat
Node(position=206, is_leaf=False, m=1, keys=[26, -1], leftchild=[173, -1], rightchild=74)
Node(position=173, is_leaf=False, m=1, keys=[10, -1], leftchild=[140, -1], rightchild=41)
Node(position=140, is_leaf=True, m=1, keys=[9, -1], leftchild=[87632, -1], rightchild=41)
Node(position=41, is_leaf=True, m=2, keys=[10, 20], leftchild=[84382, 57455], rightchild=239)
Node(position=74, is_leaf=False, m=2, keys=[68, 86], leftchild=[239, 107], rightchild=8)
Node(position=239, is_leaf=True, m=2, keys=[26, 37], leftchild=[1290832, 2132], rightchild=107)
Node(position=107, is_leaf=True, m=2, keys=[68, 84], leftchild=[97321, 431142], rightchild=8)
Node(position=8, is_leaf=True, m=2, keys=[86, 87], leftchild=[67945, 984796], rightchild=-1)
Header -> b: 3, Root Node Position: 206

```

python bptree.py -ph index.dat 을 이용하시면 전체 트리 구조를 확인하실 수 있습니다.

delete 86



```

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.097s)
python bptree.py -d index.dat delete.csv

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.085s)
python bptree.py -ph index.dat
Node(position=206, is_leaf=False, m=1, keys=[26, -1], leftchild=[173, -1], rightchild=74)
Node(position=173, is_leaf=False, m=1, keys=[10, -1], leftchild=[140, -1], rightchild=41)
Node(position=140, is_leaf=True, m=1, keys=[9, -1], leftchild=[87632, -1], rightchild=41)
Node(position=74, is_leaf=False, m=2, keys=[10, 20], leftchild=[84382, 57455], rightchild=239)
Node(position=41, is_leaf=False, m=2, keys=[68, 87], leftchild=[239, 107], rightchild=8)
Node(position=239, is_leaf=True, m=2, keys=[26, 37], leftchild=[1290832, 2132], rightchild=107)
Node(position=107, is_leaf=True, m=2, keys=[68, 84], leftchild=[97321, 431142], rightchild=8)
Node(position=8, is_leaf=True, m=1, keys=[87, -1], leftchild=[984796, -1], rightchild=-1)
Header -> b: 3, Root Node Position: 206

```

delete 9

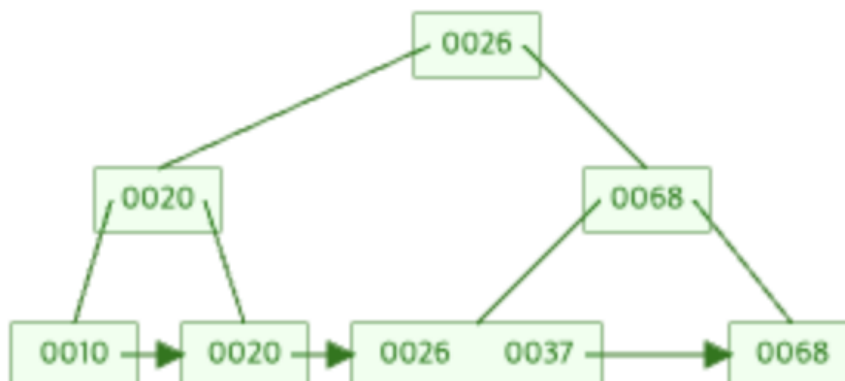


```

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.081s)
python bptree.py -d index.dat delete.csv

base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.08s)
python bptree.py -ph index.dat
Node(position=206, is_leaf=False, m=1, keys=[26, -1], leftchild=[173, -1], rightchild=74)
Node(position=173, is_leaf=False, m=1, keys=[20, -1], leftchild=[140, -1], rightchild=41)
Node(position=140, is_leaf=True, m=1, keys=[10, -1], leftchild=[84382, -1], rightchild=41)
Node(position=41, is_leaf=True, m=1, keys=[20, -1], leftchild=[57455, -1], rightchild=239)
Node(position=74, is_leaf=False, m=2, keys=[68, 87], leftchild=[239, 107], rightchild=8)
Node(position=239, is_leaf=True, m=2, keys=[26, 37], leftchild=[1290832, 2132], rightchild=107)
Node(position=107, is_leaf=True, m=2, keys=[68, 84], leftchild=[97321, 431142], rightchild=8)
Node(position=8, is_leaf=True, m=1, keys=[87, -1], leftchild=[984796, -1], rightchild=-1)
Header -> b: 3, Root Node Position: 206
  
```

delete 87,84



```
base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.086s)
```

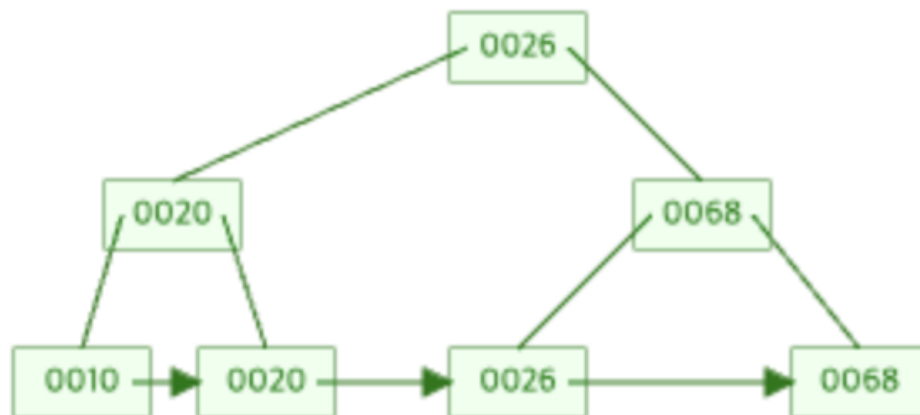
```
python bptree.py -d index.dat delete.csv
```

```
base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.084s)
```

```
python bptree.py -ph index.dat
```

```
Node(position=206, is_leaf=False, m=1, keys=[26, -1], leftchild=[173, -1], rightchild=74)
Node(position=173, is_leaf=False, m=1, keys=[20, -1], leftchild=[140, -1], rightchild=41)
Node(position=140, is_leaf=True, m=1, keys=[10, -1], leftchild=[84382, -1], rightchild=41)
Node(position=41, is_leaf=True, m=1, keys=[20, -1], leftchild=[57455, -1], rightchild=239)
Node(position=74, is_leaf=False, m=2, keys=[68, -1], leftchild=[239, -1], rightchild=8)
Node(position=239, is_leaf=True, m=2, keys=[26, 37], leftchild=[1290832, 2132], rightchild=8)
Node(position=8, is_leaf=True, m=1, keys=[68, -1], leftchild=[97321, -1], rightchild=-1)
Header -> b: 3, Root Node Position: 206
```

delete 37



```
base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.084s)
```

```
python bptree.py -ph index.dat
```

```
Node(position=206, is_leaf=False, m=1, keys=[26, -1], leftchild=[173, -1], rightchild=74)
Node(position=173, is_leaf=False, m=1, keys=[20, -1], leftchild=[140, -1], rightchild=41)
Node(position=140, is_leaf=True, m=1, keys=[10, -1], leftchild=[84382, -1], rightchild=41)
Node(position=41, is_leaf=True, m=1, keys=[20, -1], leftchild=[57455, -1], rightchild=239)
Node(position=74, is_leaf=False, m=2, keys=[68, -1], leftchild=[239, -1], rightchild=8)
Node(position=239, is_leaf=True, m=1, keys=[26, -1], leftchild=[1290832, -1], rightchild=8)
Node(position=8, is_leaf=True, m=1, keys=[68, -1], leftchild=[97321, -1], rightchild=-1)
Header -> b: 3, Root Node Position: 206
```

big data

```
base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.075s)
python bptree.py -c index.dat 10
```

```
base ~/desktop/univ./4-2/database/btree git:(main)±14 (1.404s)
python bptree.py -i index.dat input.csv
```

```
base ~/desktop/univ./4-2/database/btree git:(main)±14 (0.107s)
python bptree.py -r index.dat 9000 10000
```

```
9000,427828
9001,914573
9002,251979
9003,301232
9004,340356
9005,479080
9006,560313
9007,775082
9008,39456
9009,722090
9010,580681
9011,737411
9012,60632
9013,267554
9014,195604
9015,2368
9016,7112
9017,794454
9018,440160
```

큰 data 에 대해서도 10만개, 100만개 insert 해보았고 정상적으로 동작하는 것을 확인하였습니다.